Python for Data Science

Week(4): Functions & Modular Programming

GitHub Repository

Moodle

Dr. Ghadeer Mobasher

Agenda

1. **Purpose of Functions**
   Functions

2. **Function Parameters & Return Values**
   Parameters and return values

3. **Functions with Default Parameters**
   Default Parameters

4. **Variable Scope**
   Local and Global variables

5. **Functions passing an Arbitary Number of Arguments**
   Arbitary Number of Arguments (Positional vs Keyword)

6. **Lambda Expression**
   Anonymous functions for short operations

7. **Putting it all together**
   Final Mini Challenge: University Lab Submission Tracking System

Functions

Functions are reusable blocks of code that perform a specific task. They help:

- Avoid *code repetition*
- Make code *modular and readable*
- Allow *easy debugging and testing*

Example:

```
def greet(name):
    """Print a greeting message for the given name."""
    print(f"Hello, {name}!")

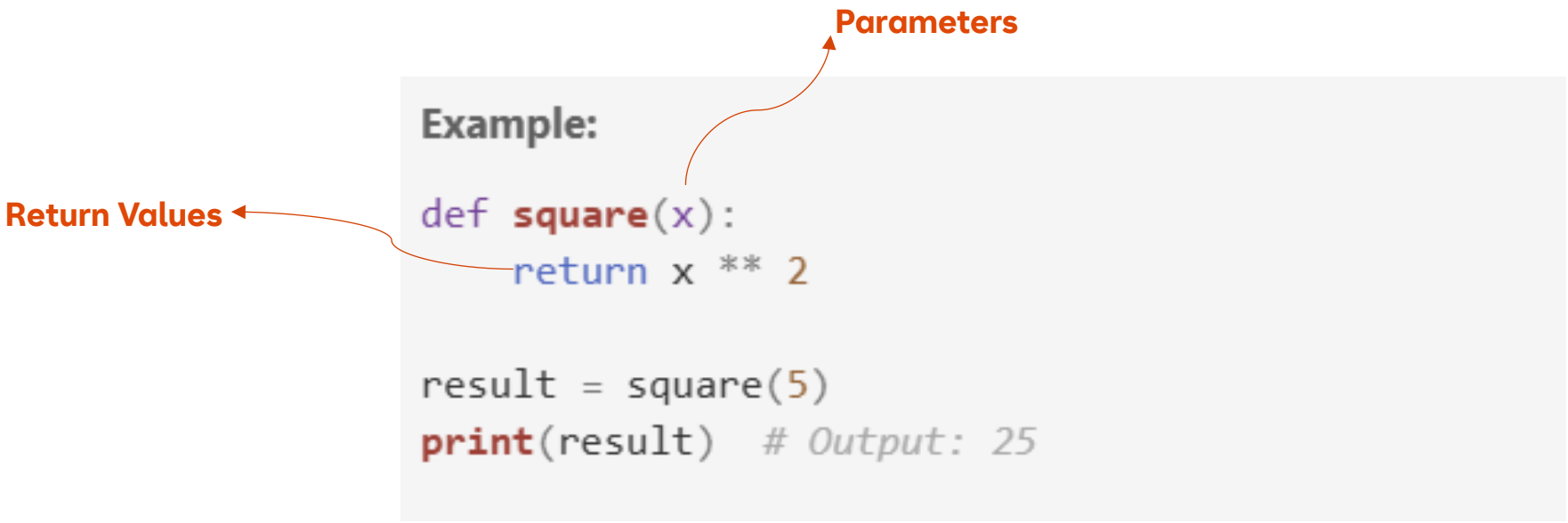
greet("Alice")
greet("Bob")
```



What is the function name? Does this function return any value?

Parameters and Return Values

- **Parameters** are inputs to a function.
- **Return values** allow a function to produce output that can be stored or used later.



Void vs Return Functions

```
# define a function to generate and print a full name
def generate_full_name():
    # assign the first name
    first_name = 'Tom'

    # assign the last name
    last_name = 'Hanks'

    # define a space character to separate names
    space = ' '

    # concatenate first name, space, and last name to form the full name
    full_name = first_name + space + last_name

    # print the full name
    print(full_name)

# call the function to execute it
generate_full_name()
```

Void Function

Void vs Return Functions

```python
# define a function that generates and returns a full name
def return_generate_full_name():
    # assign the first name
    first_name = 'Tom'

    # assign the last name
    last_name = 'Cruise'

    # define a space character to separate names
    space = ' '

    # concatenate first name, space, and last name to form the full name
    full_name = first_name + space + last_name

    # return the full name instead of printing it
    return full_name

# call the function and print the returned full name
print(return_generate_full_name())
```

Return Function

Default Parameters

Functions can have **default values** for parameters if no argument is provided.

Example:




```
def greet(name="Guest"):
    print(f"Hello, {name}!")

greet("Alice")  # Hello, Alice!
greet()         # Hello, Guest!
```

Variable Scope

Is only accessible within that function's scope

- **Local variables** exist inside a function.
- **Global variables** exist outside and can be accessed anywhere, but modifying them inside a function requires **global** keyword.

Example:

```
x = 10  # global

def modify():
    y = 5  # local
    print("Local y:", y)

modify()
print("Global x:", x)
```



```
Local y: 5
Global x: 10
```

Modifying Global Variables inside a Function

To modify a glo

```
counter = 0

def increme
    global
    counter

increment()
print(count
```

0.0s

```
1
```

```
counter = 0  # global variable

def increment():
    counter += 1

increment()
print(counter)  # output: 1
```

0.3s

```
---------------------------------------------------------------------------
UnboundLocalError                         Traceback (most recent call last)
Cell In[15], line 6
      3 def increment():
      4     counter += 1
----> 6 increment()
      7 print(counter)  # output: 1

Cell In[15], line 4
      3 def increment():
----> 4     counter += 1

UnboundLocalError: cannot access local variable 'counter' where it is not associated with a value
```

Arbitrary Number of Arguments

In Python, an **arbitrary number of arguments** allows a function to accept a variable number of inputs rather than a fixed number.

Arbitrary Positional Arguments: `*args`

The `*args` syntax allows a function to accept any number of positional arguments. These are stored as a **tuple.**

```
def add_numbers(*args):
    total = 0
    for num in args:
        total += num
    return total
print(add_numbers(1, 2, 3))       # output: 6
print(add_numbers(5, 10, 15, 20)) # output: 50
```

Use `*args` when you **don't know in advance how many positional arguments will be passed** to a function.

Arbitrary Keyword Arguments

Arbitrary Keyword Arguments: `**kwargs`
The `**kwargs` syntax allows a function to accept any number of keyword arguments. These are stored as a **dictionary.**

```python
# define a function that accepts an arbitrary number of keyword arguments
def display(**kwargs):
    print(kwargs)

display(name="Alice", age=25, city="Berlin")
```



```
{'name': 'Alice', 'age': 25, 'city': 'Berlin'}
```

Arbitrary Keyword Arguments

```python
# define a function that accepts an arbitrary number of keyword arguments
def print_info(**kwargs):
    # iterate over each key-value pair in the dictionary
    for key, value in kwargs.items():
        # print the key and its corresponding value
        """name : Alice
           age : 25
           city : Berlin"""
        print(key, ":", value)
# call the function with keyword arguments
print_info(name="Alice", age=25, city="Berlin")
# 'name', 'age', and 'city' are passed as keyword arguments
# inside the function, they are stored in the dictionary 'kwargs'
```



```
name : Alice
age : 25
city : Berlin
```

Use `**kwargs` when you want a function to accept an arbitrary number of keyword arguments.

Combining Both Arguments

```python
def example_function(*args, **kwargs):
    print("Positional arguments:", args)
    print("Keyword arguments:", kwargs)

example_function(1, 2, 3, name="Bob", age=30)
```

```
Positional arguments: (1, 2, 3)
Keyword arguments: {'name': 'Bob', 'age': 30}
```

In **Week4.ipynb**, check all sections just before "Section 5. Lambda Expression"

In **Week4_Exercise.ipynb**, you have 40 minutes to solve "**Problems 1 &2**" .

First, download this Colab Notebook and work via VS or Jupyter Notebook

Don't forget to split your screen & copy-paste given data / function names.

Pause / Break

See you in 15 minutes ☺!
Please be on time.

Lambda Expression

- Anonymous functions for short operations.
- Typically one-liner with `lambda arguments: expression`

Example:

```python
square = lambda x: x ** 2
print(square(6))  # Output: 36

add = lambda a, b: a + b
print(add(3, 5))  # Output: 8
```

Self-invoking Lambda Function

```
# option 2: define the same functionality using a lambda (anonymous function)
# lambda functions are concise, single-expression functions without a formal name
add_two_nums = lambda a, b: a + b

# call the lambda function and print the result
print(add_two_nums(2, 3))


# self-invoking lambda function
# define and immediately execute the lambda with arguments (2, 3)
print((lambda a, b: a + b)(2, 3))
```

Power Lambda Function (call the first function, then immediately call whatever it returns)

```
# explanation:
'''
- first, power(2) returns a lambda function that raises 2 to any power
- then, this returned lambda function is immediately called with the argument 3, computing 2 ** 3
- the result (8) is assigned to the variable cube
'''


# define a function that returns another function (closure)
def power(x):
    # return a lambda function that takes n and computes x raised to the power n
    return lambda n: x ** n

# call power(2) to create a function that raises 2 to a given power,
# then immediately call it with 3
cube = power(2)(3)
# print the result (expected: 8)
print(cube)
```

Call 1 → Call 2

```
# Expanded (identical behavior):
raise_2_to = power(2)    # Step 1: get back  lambda n: 2 ** n
cube = raise_2_to(3)     # Step 2: call it with n=3  →  8

# Collapsed onto one line (what you have):
cube = power(2)(3)       # Same thing, just chained
```

In **Week4.ipynb**, check "Section 5. Lambda Expression"

In **Week4_Exercise.ipynb**, you have 40 minutes to solve "**Problems 3 &4**" .

University Lab Submission Tracking System

Final Mini Challenge: University Lab Submission Tracking System

Scenario

You are asked to implement a simple system that tracks student lab submissions for a university module. The system should clean the data, remove duplicate submissions, classify student engagement, compute weekly statistics, and present the results clearly.

This challenge combines concepts from weeks 1, 2, 3, and 4:

- control flow and logical reasoning
- conditional statements and decision structures
- comparison and logical operators
- loops and iteration
- basic list comprehensions
- lists, tuples, dictionaries, and sets
- nested data structures
- functions, parameters, return values, and variable scope
- default parameters
- lambda expressions
- reusable and modular code

Given Data

Each tuple contains:

1. a student ID



In **Week4_Exercise.ipynb**, you have 60 minutes to solve “Mini Challenge”.

What did we learn today?

Purpose of Functions

Functions

Function Parameters & Return Values

Parameters & Return Functions

Variable Scope

Global and Local Variables

Functions with Default Parameters

Default Parameters

Functions passing an Arbitary Number of Arguments

Arbitary Number of Arguments

Lambda Expression

Anonymous functions for short operations

Coming Up : Week (5)

So far, we have organised our code using functions and made it more modular. How can we now store our data in files, work with different file formats, and handle possible errors when reading and writing data?

• File systems and directory structure • Reading and writing text and CSV files
• Structured data formats • Error handling and exception concepts
• Basic parsing principles

Any Questions?

See you next time ☺

Dr. Ghadeer Mobasher
ghadeer.mobasher@srh-hochschulen.de
https://www.linkedin.com/in/ghadeermobasher/
LGS 6, 2. OG, Arc 213